Generative Parametric Design of Gothic Window Tracery

Sven Havemann
Computer Graphics Group
TU Braunschweig,Germany
s.havemann@tu-bs.de

Dieter Fellner
Computer Graphics Group
TU Braunschweig, Germany
d.fellner@tu-bs.de

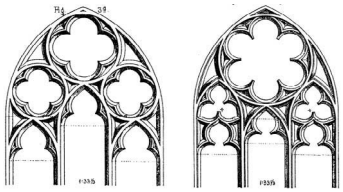


Fig. 1. Examples of window tracery from the high gothic period.

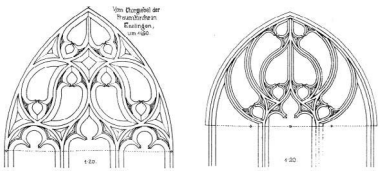


Fig. 2. Examples of window tracery from the late gothic period.

***Abstract*— Gothic architecture, and especially window tracery, exhibits quite complex geometric shape configurations by combining only a few basic geometric patterns, namely circles and straight lines. They are combined using only a limited set of operations, such as intersection, offsetting, and extrusions. The reason lies in the process how these objects have been physically realized, i.e., through construction with compass and ruler. Consequently, Gothic architecture is a great, although challenging, domain for parametric modeling.**

We present some principles of this long-standing domain, together with some delicate details, and show how the constructions of some prototypic Gothic windows can be formalized using our Generative Modeling Language (GML). The emphasis of this procedural approach is on modularization, so that complex configurations can be obtained from combining elementary constructions. Different combinations of specific parametric features can be grouped together, which leads to the concept of styles. They permit to differentiate between the basic shape and its appearance, i.e., a particular ornamental decoration.

This leads to an extremely compact representation for a whole class of shapes, which can nevertheless be quickly evaluated to obtain a connected manifold mesh of a particular window instance. The resulting mesh may also contain free-form surface parts, represented as subdivision surfaces.

I. The construction of a Gothic Window

The basic pattern in Gothic Architecture is the *pointed arch*. Its geometric construction is based on the intersection of two circles. The circles are tangent continuous to the sides of an arch or a window, given as two parallel vertical line segments. Consequently the midpoints m_L and m_R of the circles lie on the horizontal line through the upper segment endpoints p_L and p_R. The pointed arch is symmetric, so both circles have the same radius $r = \mathrm{dist}(p_L, m_R) = \mathrm{dist}(p_R, m_L)$.

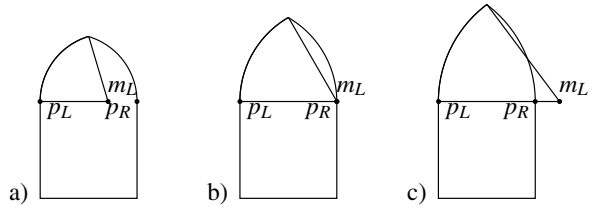


Fig. 3. Gothic arch with varying excess parameters: a) Four-centered (0.70), b) equilateral (1.0), and c) pointed arch (1.25).

We call the ratio $r/dist(p_L, p_R)$ the *excess* of the arch. When the excess is 1.0, the circle midpoints coincide with the upper segment endpoints. Together with the circle intersection, they form an equilateral triangle. This is the standard pointed arch, also called *equilateral arch*. With an excess > 1.0, the circles intersect at a sharper angle, and this is what is actually called a *pointed arch*. When the excess is < 1.0, the arch is not so high, resulting in a *four-centered arch*. The extreme case is a round arch with an excess of 0.5, where m_L, m_R coincide in the middle between p_L, p_R. Examples are shown in Fig. 3.

A. The Prototype Window

The focus of this paper is to demonstrate how some construction rules for Gothic windows can be mapped to a parametric modeling language. We chose one typical window from the High Gothic period as our prototype, as it exhibits the main shape features, the "shape vocabulary", that was subsequently refined and varied in the later period (Fig. 2).

This prototype window is shown in Fig. 4. It has two sub-windows which are also pointed arches. They have the same form, and most often also the same excess as the larger arch. So the excess parameter is one high-level parameter of the window. The window shown has excess 1, so that the midpoints of the circular arcs and the segment endpoints p_L, p_R coincide. They are also the midpoints of the circular arcs from the subwindow's arches. The right window shows another degree of freedom: Often the sub-windows are set down a bit.

Proceedings of the Shape Modeling International 2004 (SMI'04)
0-7695-2075-8/04 $20.00 © 2004 IEEE

IEEE COMPUTER SOCIETY

Authorized licensed use limited to: New York University. Downloaded on October 16,2026 at 18:04:26 UTC from IEEE Xplore. Restrictions apply.

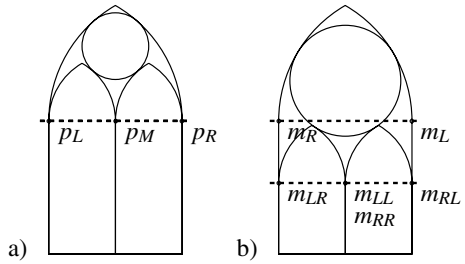


Fig. 4. Geometry of the prototype window

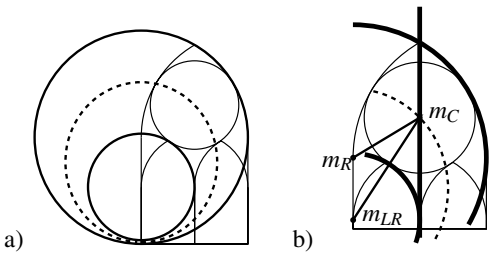


Fig. 5. Determining the midpoint and radius of the rosette's circle

.

B. Adding a Circular Rosette

Gothic windows exhibit a great variety in filling the space between the big arch and the sub-arches. This delicate stone work, which significantly evolved over the centuries, is called *window tracery*. In our prototype window, we chose to fill this space with a circular rosette. The midpoint of the rosette circle has the same distance to e.g. the right arc of the big arch and the right arc of the left sub-arch, as indicated by the dotted line in Fig. 5. This line is an ellipse, as for all of its points m the sum $\text{dist}(m,m_R)+\text{dist}(m,m_{LR})$ must be constant. As the window is symmetric, the midpoint m_C of the rosette circle can be found by intersecting the ellipse with a line.

C. A Rosette Window with Multiple Foils

A common way to fill a circular region is by an rosette with multiple foils, for example a trefoil or a quatrefoil. We consider two variants of rosettes, namely with round and with

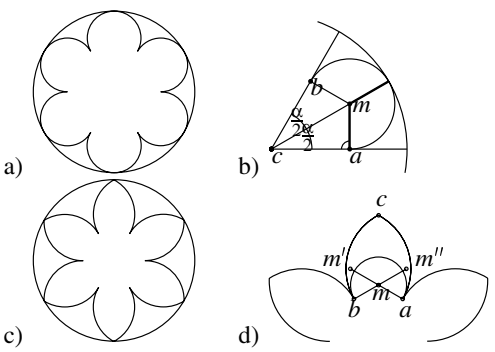


Fig. 6. Construction of a rosette with six rounded foils, so $\alpha=\frac{2\pi}{6}$.

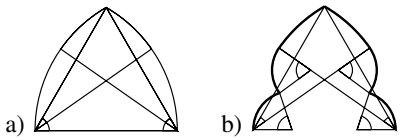


Fig. 7. Pointed trefoil obtained from pointed arch.

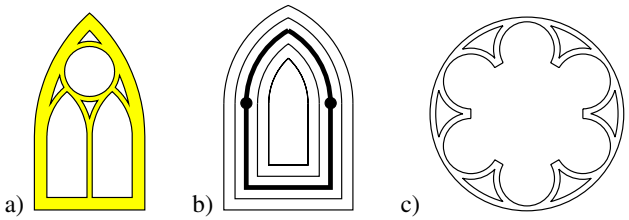


Fig. 8. a) The regions are shrunk to embed them in a common border plane. b) The offset operation changes the excess of a pointed arch, but keeps the circle midpoints constant. c) Exactly the same operation (and the same GML operator) can be used for rosette windows as well.

pointed foils. The geometry is fairly straightforward: Given the number n of foils in a unit circle, the radius r of the round foils is computed as shown in Fig. 6 b). Consider the tangent from center c to the circle (m,r). The distance from c to m is $1-r$, so the length of the perpendicular from m to the tangent is $(1-r)\sin\frac{\alpha}{2}$, which is supposed to be r as well. This equation gives $r=\sin\frac{\alpha}{2}/(1+\sin\frac{\alpha}{2})$. The perpendicular feet a and b are the endpoints of the circular arc that is rotated and copied n times to make up the rosette.

The pointed foils can be obtained from the round foils by symmetric displacement of the midpoint m along the lines (a,m) and (b,m) (see Fig. 6), together with the intersection point c of these circles. The amount of displacement determines the pointedness.

D. Further Refinements

Circular arcs can be combined very flexibly. Both corners and tangent continuous joints can be obtained from quite elementary geometric constructions. The pointed trefoil in Fig. 7 for instance is obtained from a pointed arch by replacing each of the lower parts of the arc by a pair of smaller arcs. Tangent continuity is obtained by choosing the midpoint of the next arc on the line through mid- to endpoint of the given arc. This example shows the source of the great variability of geometric patterns in Gothic architecture.

E. Offset Curves and Profiles

The sub-arches and the rosette circle partition the window into disjoint regions, defining the basic window structure. It is further refined by adding a profile around the actual window holes to emphasize the shape, and, especially in the late Gothic period, also by adding more and more sub-structures, again composed of lines and circular arcs. Often the region border curve is offset by a certain distance so that a thin contiguous border plane results, as shown in yellow in Fig. 8 a.

One great thing about the circle is that its offset is again a circle. Also the offset of a connected sequence of circular arcs

IEEE COMPUTER SOCIETY

Authorized licensed use limited to: New York University. Downloaded on October 16,2026 at 18:04:26 UTC from IEEE Xplore. Restrictions apply.

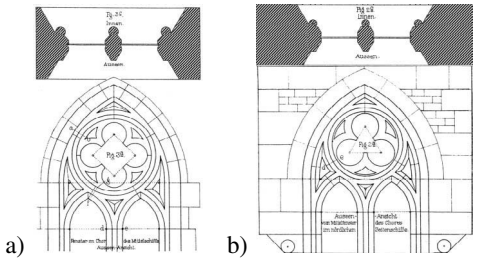


Fig. 9. Window profiles.

and line segments is again a sequence of such elements (if it exists). If the sequence contains corners, e.g. arcs joining in a common intersection of the respective circles, the intersection of the offset circles must be computed for obtaining the offset sequence. Simple scaling is not sufficient: The offset of a pointed arch has a different excess than the original arch, see Fig. 8 b).

F. Appearance: Profiles

The fascinating and impressive three-dimensionality of Gothic windows is achieved by *profiles* that give depth to the two-dimensional geometric figures, as in Fig. 9 from Egle [1]. These profiles are swept along the region border curves, with the profile plane orthogonal to the tangent of the curve. At discontinuos corner points the sweep is continued onto the bisector plane, the plane spanned by the bisecting line of the corner angle and the normal of the 2D construction plane. In more general cases though, i.e. when the curve is not locally symmetric to the bisector plane, the profile locally follows the *medial axis* of the curve.

II. GML Realization

The *Generative Modeling Language* (GML) is a very simple, stack-based programming language for creating polygonal meshes [2]. The language core is very similar to that of Adobe's PostScript, as concisely described in chapter 3 of the PostScript "Redbook" [3], which largely applies also to the GML. The GML doesn't have PostScript's extensive set of operators for typesetting, though. Instead, it provides quite a number of operators for three-dimensional modeling, from low-level Euler operators for mesh editing, to high-level modeling tools, such as different forms of extrusions, and operators to convert back and forth between polygons and mesh faces.

A. The Pointed Arch

A circular arc can be specified by start- and endpoint, and the midpoint of the circle, like in Fig. 6 b). In the GML, this is an array of three points, written `[ a m b ]`. To distinguish between the two possible arcs from a to b, we specify a normal vector n with the convention that the arc is always CCW oriented when the normal points to the viewer. Now given two circles (m_L, r) and (m_R, r) as in Fig. 3, the circle intersection is computed by an operator that expects midpoints and radii and the plane normal on the stack: Every GML operator pops its arguments from the stack, computes one or more results, and pushes them on the stack. In this case, the intersection points above and below the line between m_0 and m_1 are pushed on the stack, in this order. As we're only interested in one of them, the other one is popped. The two arcs are assembled with just one more line of code:

```
mL r mR r nrml intersect_circles pop !q
[ :q mL pL ] [ pR mL :q ]
```

The intersection is stored in a named register `q`. One drawback of stack-based programming is that can be very tedious to keep track of the items and their order. Alternatively, variables can be stored in dictionaries – but searching through the dictionary stack may be slow. As a third alternative, the GML provides *named registers* for local variables: Values can be set using `!q` and retrieved using `:q`. This encourages a more "procedural" programming style, while still retaining the flexibility of the stack for parameter passing.

```
{ !nrml !offset !excess !pR !pL
  :pR :pL :excess line_2pt !mR
  :pL :pR :excess line_2pt !mL
  :pR :mR dist :offset sub !r
  :mL :r :mR :r :nrml intersect_circles pop !q
  [ :q :mL   :pL :pR :offset move_2pt ]
  [ :pR :pL :offset move_2pt   :mR :q ]
} /compute-arcs exch def
```

The second line of code computes the midpoint m_R of the right arch as $p_R + excess(p_L - p_R)$. The radius r of both circles is computed as the distance from p_R to m_R minus the desired offset. The circular arcs are assembled "inline": An open bracket `[` is just a literal symbol that is put on the stack. The `move_2pt` operator moves a point into the direction of another point by a specified amount of units. The closed `]` just makes the interpreter create an array from stack elements until it finds the `[`. The family of pointed arches in Fig. 8 b) was created using this function by varying the offset parameter.

B. Creating a Closed Polygon

The resulting two arcs can be seen as a high-level representation of a pointed arch. In order to create a polygonal mesh, the arcs are converted to polygons: The `circleseg` operator expects an arc, a normal vector, an integer resolution, and a mode flag on the stack, and pushes a polygon. The advantage of the stack is the possibility of *operator chaining*: The arc is already on the stack, so only the other parameters must be provided. To also polygonize the other arc, the topmost stack elements are swapped using `exch`, and the two arrays are concatenated into one polygon using the `arrayappend` operator:

```
nrml 4 1 circleseg exch
nrml 4 1 circleseg arrayappend
[ bL dup bR dup ] arrayappend
```

The order is always very crucial when operating on the stack: The arcs were pushed in order left, right, and they are processed right, left, to create a combined CCW oriented

IEEE COMPUTER SOCIETY

Authorized licensed use limited to: New York University. Downloaded on October 16,2026 at 18:04:26 UTC from IEEE Xplore. Restrictions apply.

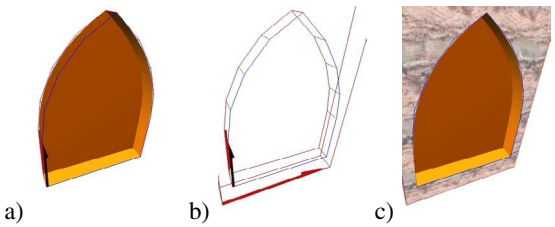


Fig. 10. Creating a window in a wall: a) The arch is inverted by negative extrusion. The backface (culled, not rendered) is closer and CW oriented (black halfedge). b) A wall with its front face in the same geometric plane as the arch's backface. The backface (black halfedge) is made a ring of the wall's front face (halfedge at bottom) using the `killFmakeRH` Euler operator. c) The tesselation of the wall respects the ring just created and trims it out.

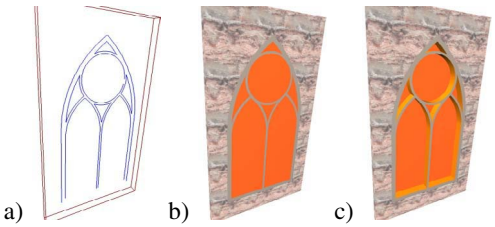


Fig. 11. a) The execution of `gothic-window` yields eight polygons on the stack defining the structure of the prototype window. b) The seven interior polygons are embedded in the face created from the outer arch. c) Simple negative etrusion of the interior regions.

polygon. Note that it contains the tip of the arch twice, as the last point of the right arc and the first point of the left arc. The bottom corners b_L, b_R are also duplicated and appended, so that the result is a single polygon.

C. Creating a Polygonal Mesh

To create a mesh from this polygon, it is turned into a double-sided face, negatively extruded, and set into a wall:

```
5 poly2doubleface (0,-0.3,5) extrude
exch eWall killFmakeRH
```

The `poly2doubleface` operator expects a polygon and a mode flag, and it returns a handle to a halfedge of the newly created face. Halfedges are an integral data type in the GML. Besides a halfedge, the `extrude` operator expects a 3D point $(dx, dy, mode)$, where dy is the extrusion depth and dx is an offset factor for shrinking or enlarging. The *mode* flags of these operations specify the *sharpness* of the created edges, cf. Fig. 10 b), where *sharp edges* are shown in red, and *smooth edges* are green. It also shows the halfedge returned by `extrude` as the red half-arrow in the lower left.

The underlying mesh representation is the *Combined BRep* (CBRep) [4], where each edge has a binary sharpness flag. These flags determine which tesselation method to use for a face: Faces with at least one smooth edge are *smooth faces*, treated as Catmull/Clark subdivision surfaces. Another feature of Combined BReps is that faces may have rings to trim out holes. One way to create a hole is by *negative extrusion*, so that the model appears inverted when backface culling is active. The hole is created using the *Euler operator* `killFmakeRH`, which reads "kill Face, make Ring Hole" (Fig. 10 a,c).

D. The Prototype Window in GML

The GML representation of the prototype Gothic window is of course a function `gothic-window` that expects the essential parameters as described in section I-A on the stack: The left and right base points of the arch, the height of the vertical segments of the lower part of the window, the excess of the arch, the plane normal, the vertical offset of the sub-arches (cf. Fig. 4), and the outer and inner border thicknesses. The following call produces the eight polygons that are shown in Fig. 11 a):



```
(-1,0,2) (1,0,2) 4.0 1.25 (0,-1,0) 0.2 0.1 0.05
gothic-window
```

This function uses only operators and techniques that were presented so far. The only additional operator needed is `intersect_line_ellipse`, to compute the midpoint of the circle in the top from two points specifying the line, the two focal points of the ellipse, and its radius.

An interactive demo with all the variations shown above together in one GML file of 25 KB can be downloaded from http://www.generative-modeling.org.

References

[1] von Egle and Fiechter, *Gotische Baukunst*, reprint from 1905 ed., ser. Baustil- und Bauformenlehre. Verlag Th. Schäfer, Hannover, Germany, 1996, vol. Bd. 3.

[2] Havemann. (2003, Aug.) The gml homepage. [Online]. Available: http://graphics.tu-bs.de/gml

[3] Adobe Systems Inc., *PostScript Language Reference Manual*, 3rd ed. Addison-Wesley, 1999.

[4] Havemann and Fellner, "Progressive combined breps – multi-resolution meshes for incremental real-time shape manipulation," *Computer Graphics Forum*, submitted for publication, also available as technical report TUBSCG-2003-01, Institute of ComputerGraphics, TU at Brunswick, Germany.

IEEE COMPUTER SOCIETY

Authorized licensed use limited to: New York University. Downloaded on October 16,2026 at 18:04:26 UTC from IEEE Xplore. Restrictions apply.